Online Bayesian Optimization for Real-Time Adaptive MPC in Autonomous Agent Navigation

Lorenzo Ortolani, Gabriel Voss, Francesco Dorati, Tommaso Felice Banfi
TALOS Robotics
Milan, Italy
{lorenzo.ortolani, gabriel.voss, francesco.dorati, tommasofelice.banfi}@talosrobotics.ai

***Abstract*—Real-time autonomous navigation in unknown, dynamically changing environments is a fundamental challenge for mobile agents across a wide spectrum of robotic platforms. We present a general-purpose navigation framework that tightly couples reactive rolling-horizon path planning with adaptive nonlinear Model Predictive Control (MPC), requiring no prior map knowledge. The system reconstructs a Gaussian occupancy grid from raw LiDAR observations at each planning cycle and runs A* search to generate collision-free reference trajectories tracked by a CasADi/IPOPT MPC formulation equipped with a differentiable sigmoid obstacle barrier. To address the inherent sensitivity of MPC performance to cost-function parameters, we propose a dual-stage adaptation strategy: *offline* Bayesian Optimization via Tree-structured Parzen Estimators (TPE) for principled pre-tuning across a diverse training suite, and *online* Bayesian Optimization that incrementally refines parameters during deployment via real-time performance feedback and Gaussian Process (GP) posterior updates. Additionally, a self-supervised Dynamic Graph Convolutional Neural Network (DGCNN) encoder extracts compact, rotation-invariant scene embeddings from LiDAR point clouds, enabling environment-conditioned MPC parameter adaptation without manual terrain annotation. While the framework is agnostic to robot morphology, we evaluate it on the Unitree Go2 quadruped in Gazebo simulation across five distinct environments and validate hardware deployment on the physical robot, confirming that simulation-tuned parameters transfer with comparable performance. The full system achieves up to 98% navigation success rate with average cross-track error below 0.10 m in open environments and generalizes to previously unseen dynamic and complex indoor scenarios with 82–85% success rate.**

***Index Terms*—Autonomous Navigation, Model Predictive Control, Bayesian Optimization, Graph Neural Networks, Legged Robots, Point Cloud Encoding, Self-Supervised Learning, Rolling-Horizon Planning**

I. Introduction

Autonomous navigation in cluttered, a priori unknown environments is a cornerstone capability for mobile robots deployed in real-world settings — from industrial inspection [**?**] and search-and-rescue [**?**] to household assistance [**?**] and last-mile delivery [**?**]. The challenge is compounded for legged platforms, whose complex contact dynamics and non-holonomic constraints at the body level demand particularly smooth and dynamically consistent motion commands from the navigation layer. Classical decoupled architectures — in which a global planner produces a geometric path and a low-level controller tracks it independently — fail to capture the coupling between planning and dynamic feasibility [**?**], leading to jerky execution, frequent constraint violations, and poor adaptability to sudden environmental changes.

Model Predictive Control (MPC) provides a principled framework for simultaneous path following and constraint satisfaction over a receding planning horizon [**?**]. Its applicability to real-time robotics is contingent on two practical requirements that have historically limited deployment: (i) a computationally efficient, always-fresh representation of the environment for obstacle avoidance, and (ii) a well-calibrated set of cost-function weights that balance tracking accuracy, control effort, and safety margins. Manual tuning of these weights is notoriously time-consuming and environment-specific, and weight sets that perform well in open spaces often produce over-cautious or oscillatory behavior in constrained corridors [**?**].

We address these limitations with a hierarchical, modular navigation stack designed as a general-purpose framework for autonomous agents and validated on the Unitree Go2 quadruped [**?**]. The system operates in a purely reactive, rolling-horizon fashion: a Gaussian occupancy grid is rebuilt from the latest LiDAR scan at every replanning tick, A* computes a collision-free local path on this grid at 10 Hz, and a nonlinear MPC tracker generates dynamically feasible velocity commands via IPOPT. On top of this base stack we contribute a two-stage parameter adaptation pipeline. In the offline phase, Bayesian Optimization via TPE sweeps the 13-dimensional MPC parameter space across a training suite of simulation environments, identifying near-optimal weight configurations through a structured multi-metric composite score. In the online phase, a Gaussian Process surrogate is updated incrementally during deployment to refine parameters in real time from observed navigation performance, with warm initialization from the offline solution. A self-supervised DGCNN encoder further augments adaptation by extracting dense, environment-specific embeddings from raw point clouds that condition MPC weights at every scan cycle, without requiring manual scene annotation.

Contributions:

- A tightly integrated, map-free rolling-horizon navigation stack combining Gaussian occupancy mapping, semantic graph construction via WildOS [1], A* planning, and nonlinear MPC with differentiable obstacle avoidance.
- An offline Bayesian Optimization pipeline via TPE with a principled five-metric composite score, systematically

Fig. 1. The Unitree Go2 quadruped during hardware navigation trials. The robot carries a 3-D LiDAR sensor and runs the full navigation stack onboard. Only a goal pose is provided as external input; all mapping, planning, and control are computed on the onboard NUC.

tuning a 13-dimensional MPC parameter vector across diverse simulation environments.

- An online Bayesian Optimization strategy leveraging GP posterior updates and Expected Improvement acquisition to adapt MPC parameters incrementally during deployment, warm-started from the offline solution.
- A self-supervised DGCNN encoder trained with contrastive InfoNCE loss that produces compact, rotation-invariant scene embeddings for real-time MPC parameter conditioning via a learned residual mapping.
- Experimental validation across five environments (three in-distribution, two unseen) and hardware demonstration on the physical Unitree Go2, confirming consistent performance gains at each adaptation stage and negligible sim-to-real degradation.

Although demonstrated on a quadruped, the framework is platform-agnostic: any mobile agent exposing a body-frame velocity interface can benefit from the proposed planning and adaptation pipeline. Fig. 1 shows the Unitree Go2 during a hardware navigation trial.

Paper organization. Section II formally states the navigation problem and reviews related work. Section III details the system architecture and all algorithmic components. Section IV describes the experimental setup, environments, and evaluation protocol. Section V reports quantitative results and ablation findings. Section VI concludes and outlines future directions.

II. Background

A. Problem Statement

Let the robot state at time step k be $\mathbf{x}_k = [p_{x,k},\, p_{y,k},\, \psi_k]^\top \in \mathcal{X} \subset \mathbb{R}^3$, where $(p_x, p_y) \in \mathbb{R}^2$ denotes the position in the world frame and $\psi \in (-\pi, \pi]$ is the heading angle. The control input is the body-frame velocity command $\mathbf{u}_k = [v_{x,k},\, v_{y,k},\, \omega_k]^\top \in \mathcal{U} \subset \mathbb{R}^3$, comprising forward velocity, lateral velocity, and yaw rate, respectively. At each time step the agent receives a LiDAR point cloud $\mathcal{P}_k = \{\mathbf{p}_i \in \mathbb{R}^3\}_{i=1}^{N_k}$ and a goal pose $\mathbf{x}_g \in \mathcal{X}$.

The navigation problem is formulated as a receding-horizon optimal control problem. At each planning cycle the agent solves:

$$\min_{\mathbf{u}_0,\ldots,\mathbf{u}_{N-1}} J(\mathbf{x}_{0:N},\, \mathbf{u}_{0:N-1},\, \mathcal{P}_k,\, \theta) \tag{1}$$

subject to:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t), \quad t = 0, \ldots, N-1 \tag{2}$$

$$d(\mathbf{x}_t, \mathcal{P}_k) \geq r_{\min}, \quad t = 0, \ldots, N \tag{3}$$

$$\mathbf{u}_t \in \mathcal{U}, \quad t = 0, \ldots, N-1 \tag{4}$$

where J is a composite cost function parameterized by $\theta \in \Theta \subset \mathbb{R}^{n_\theta}$, f is the discrete-time robot dynamics, $d(\mathbf{x}_t, \mathcal{P}_k)$ is the minimum Euclidean distance from the predicted state $\mathbf{x}_t$ to any obstacle in the current point cloud, and $r_{\min}$ is the minimum safety clearance. The objective is to drive the robot from its current pose $\mathbf{x}_0$ to the goal $\mathbf{x}_g$ while satisfying (2)–(4) in real time, without any prior knowledge of the environment map.

The performance of the MPC (1) depends critically on the parameter vector θ, which encodes cost weights for tracking accuracy, control effort, and obstacle avoidance. The meta-problem of identifying an optimal θ is formulated as the black-box optimization:

$$\theta^* = \arg\max_{\theta \in \Theta} \mathcal{J}(\theta) \tag{5}$$

where $\mathcal{J}(\theta)$ is a composite performance score obtained by running the full navigation system under parameters θ.

B. Related Work

1) Legged Robot Navigation: Navigation for legged robots has been studied extensively in the context of whole-body planning and terrain-aware locomotion [**?**]. Recent approaches integrate deep learning-based terrain estimation with classical planning [**?**] or employ model-free reinforcement learning for end-to-end navigation [**?**]. These methods either require pre-built terrain maps or suffer from limited interpretability and difficult sim-to-real transfer. Our framework abstracts leg dynamics through the CHAMP hierarchical locomotion controller [2], which exposes a body-frame velocity interface and enables the navigation layer to treat the robot as a holonomic mobile agent, dramatically simplifying planning and control design.

2) Reactive Planning and Occupancy Mapping: Reactive obstacle avoidance via occupancy grid maps is well established in mobile robotics [?]. Probabilistic grid maps [?] aggregate sensor history to maintain uncertainty-aware representations; our approach instead rebuilds the grid from scratch at every cycle to guarantee purely reactive, stale-free obstacle awareness with provably bounded memory usage. Grid-based deterministic planners such as A* are widely used for local navigation due to their predictable complexity [?]. Sampling-based alternatives (RRT* [?], Informed-RRT* [?]) offer probabilistic completeness but exhibit high variance in solve time, incompatible with our 10 Hz replanning budget. Recent adaptive variants such as APEI-RRT* [3] partially address convergence speed but still incur overhead prohibitive in cluttered environments.

3) MPC for Mobile Robot Navigation: MPC has been applied to mobile robot navigation across differential-drive platforms [?], UAVs [4], and quadrupeds [?]. Nonlinear MPC formulations handle the full kinematic model but require efficient NLP solvers; CasADi [?] with IPOPT [?] provides a suitable combination for our 3-second horizon at 10 Hz. Obstacle avoidance within MPC is commonly implemented via hard distance constraints [?] or smooth barrier functions [?]; we adopt the latter through a differentiable sigmoid penalty evaluated on the nearest LiDAR returns, which preserves NLP sparsity and IPOPT convergence properties.

4) Bayesian Optimization for Robotics: Bayesian Optimization (BO) has become the standard method for black-box hyperparameter tuning in robotics [?]. GP-based BO with Expected Improvement (EI) or Upper Confidence Bound (UCB) acquisition functions has been applied to locomotion gait tuning [?], controller gain selection [?], and motion planning configuration [?]. The COAt-MPC framework [5] provides performance-guaranteed BO for MPC parameter optimization, while [6] analyzes the theoretical properties of TPE, the acquisition strategy we adopt for its efficiency in high-dimensional mixed spaces. Online BO for incremental parameter adaptation during deployment has been studied in [?], though primarily in structured settings; we extend this to legged robot navigation in unstructured environments with a warm-started GP surrogate.

5) Environment Representation Learning: Learned 3-D representations have been explored for terrain classification [?], traversability estimation [?], and place recognition [?]. PointNet [?] and its hierarchical extension PointNet++ [?] established permutation-invariant architectures for point cloud processing. DGCNN [?] introduced dynamic edge convolutions that recompute the k-NN graph in feature space at each layer, yielding richer multi-scale representations. Contrastive self-supervised learning — particularly SimCLR [?] and PointContrast [?] — has demonstrated that discriminative representations can be learned from unlabeled data through view-invariance objectives, making them directly applicable to our setting where no environment class labels are available.

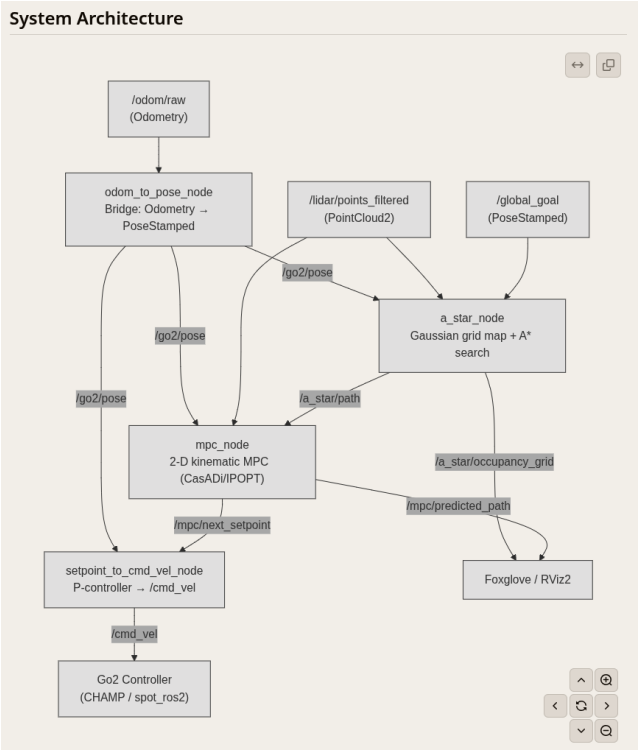

Fig. 2. System architecture. Solid arrows indicate primary data flow. The GNN encoder (dashed box) conditions the MPC parameter vector θ on the current point cloud embedding; the BO pipeline (dotted box) adapts θ offline or online from composite performance scores.

III. Methodology

A. System Architecture

The proposed navigation stack is composed of five interconnected modules illustrated in Fig. 2: (i) an *Odometry Bridge* that converts raw EKF odometry into a unified pose representation; (ii) a *Mapping Module* that maintains both a reactive Gaussian occupancy grid and a persistent semantic navigation graph via WildOS [1]; (iii) a *Rolling-Horizon A* Planner* that computes collision-free paths on the occupancy grid at 10 Hz; (iv) a *Nonlinear MPC Tracker* formulated with CasADi [?] and solved with IPOPT [?], generating dynamically feasible velocity setpoints; and (v) a *Proportional Setpoint Controller* that converts MPC lookahead poses to body-frame `/cmd_vel` commands sent to the locomotion layer. All modules communicate through ROS 2 [?] topics, enabling full modularity and real-time operation at 10–20 Hz. The MPC parameter vector θ is adapted by the Bayesian Optimization pipeline, optionally conditioned on the GNN scene embedding described in Section III-H.

B. Mapping

1) Gaussian Occupancy Grid: At each replanning cycle, a square local grid of side $2h_w$ (default $h_w = 5$ m, resolution $\Delta = 0.25$ m/cell) is centered on the robot and rebuilt from scratch using only the current LiDAR point cloud $\mathcal{P}_k$. Each grid cell c encodes the probability of occupancy derived from

the minimum Euclidean distance $d_{\min}(c, \mathcal{P}_k)$ from the cell center to any LiDAR return:

$$P(c) = 1 - \Phi\left(\frac{d_{\min}(c,\, \mathcal{P}_k)}{\sigma}\right) \tag{6}$$

where $\Phi(\cdot)$ is the standard normal CDF and $\sigma = 0.05\,\text{m}$ is the Gaussian spread parameter controlling obstacle inflation. Cells with $P(c) \geq P_{\text{thresh}} = 0.45$ are treated as hard obstacles; cells below this threshold incur a soft traversal penalty in A* proportional to their occupancy probability. The grid is intentionally stateless — it is always built from the latest scan alone — guaranteeing that no stale observations contaminate the reactive planner.

2) Semantic Navigation Graph: In addition to the reactive occupancy grid, the system maintains a persistent topological-semantic navigation graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node $v \in \mathcal{V}$ corresponds to a semantically meaningful region of the environment and each edge $(u, v) \in \mathcal{E}$ represents a navigable transition weighted by the traversal cost estimated from the current occupancy grid. Node labels are assigned by WildOS [1], an open-vocabulary object search engine that maps raw sensor observations to natural-language semantic labels without class-specific training data, enabling zero-shot semantic scene understanding. As the robot explores, new nodes are instantiated when the robot enters a previously unvisited region, and the graph is updated online. This layer enables higher-level navigation behaviors — such as semantic goal specification and context-aware replanning — while the reactive Gaussian grid handles low-level obstacle avoidance in real time.

C. Rolling-Horizon Path Planning

1) Local Goal Selection: Since the occupancy grid covers only a local window, a rolling-horizon strategy is employed for distant goals. At each replanning tick the local planning target is:

$$\mathbf{x}_{\text{local}} = \begin{cases} \text{cell}(\mathbf{x}_g) & \text{if } \mathbf{x}_g \in \text{grid} \\ \text{ray}(\mathbf{x}_0 \to \mathbf{x}_g) \cap \partial\text{grid} & \text{otherwise} \end{cases} \tag{7}$$

The ray is parametrically clipped to the grid boundary, allowing the robot to advance one grid-width at a time. If the local target falls in an occupied cell, a breadth-first search (BFS) identifies the nearest free cell as a proxy, guaranteeing A* always receives a valid goal.

2) A on the Occupancy Grid:* A* is executed on an 8-connected grid with the following edge traversal cost:

$$g(n \to n') = c_{\text{move}} \cdot \Delta \cdot (1 + w_{\text{obs}} \cdot P(n')) \tag{8}$$

where $c_{\text{move}} \in \{1, \sqrt{2}\}$ for cardinal and diagonal moves respectively and w_{obs} is the soft obstacle penalty weight. The admissible Euclidean heuristic is:

$$h(n) = \Delta \cdot \sqrt{(i_x - g_x)^2 + (i_y - g_y)^2} \tag{9}$$

Cells with $P \geq P_{\text{thresh}}$ are treated as hard obstacles (infinite traversal cost), while sub-threshold cells incur the soft penalty (8), steering paths away from obstacle proximity without requiring impassability. The resulting path is resampled at uniform spacing $\Delta s = 0.20\,\text{m}$ and smoothed with a moving-average kernel of width 5 to remove grid-induced zigzag artifacts while preserving endpoints.

D. MPC Formulation

1) State, Control, and Dynamics: The robot is modeled as a holonomic kinematic agent in the plane, with state $\mathbf{x}_k = [p_x, p_y, \psi]^\top$ and control $\mathbf{u}_k = [v_x, v_y, \omega]^\top$. The discrete-time dynamics under Forward Euler integration with step $\Delta t = 0.1\,\text{s}$ are:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) = \begin{bmatrix} p_{x,k} + (v_{x,k}\cos\psi_k - v_{y,k}\sin\psi_k)\,\Delta t \\ p_{y,k} + (v_{x,k}\sin\psi_k + v_{y,k}\cos\psi_k)\,\Delta t \\ \psi_k + \omega_k\,\Delta t \end{bmatrix} \tag{10}$$

2) Objective Function: The MPC solves the following finite-horizon NLP over $N = 30$ steps (3 s horizon) at 10 Hz:

$$\begin{aligned} J = {} & \mathbf{e}_N^\top \mathbf{Q}_T \mathbf{e}_N + J_{\text{obs}}(\mathbf{x}_N) \\ & + \sum_{k=0}^{N-1} \Big[\mathbf{e}_k^\top \mathbf{Q} \mathbf{e}_k + \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k \\ & \qquad + R_{\text{jerk}} \|\mathbf{u}_k - \mathbf{u}_{k-1}\|^2 + J_{\text{obs}}(\mathbf{x}_k) \Big] \end{aligned} \tag{11}$$

where $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}_{\text{ref},k}$ is the tracking error with respect to the A*-derived reference, $\mathbf{Q} = \text{diag}(Q_{xy}, Q_{xy}, Q_\psi)$, $\mathbf{Q}_T = Q_T \cdot \mathbf{Q}$, and $\mathbf{R} = \text{diag}(R_v, R_v, R_\omega)$. The jerk term R_{jerk} penalizes rapid control changes to enforce smooth commands. The NLP is compiled once at startup using CasADi [**?**] symbolic differentiation; subsequent solves update only numeric parameter values, enabling real-time feasibility.

3) Sigmoid Obstacle Barrier: Obstacle avoidance is enforced as a smooth cost penalty through a logistic sigmoid barrier evaluated on the $M = 12$ nearest LiDAR returns $\{\mathbf{p}_j\}_{j=1}^{M}$ within a search radius of 3 m:

$$J_{\text{obs}}(\mathbf{x}_k) = \sum_{j=1}^{M} \frac{W}{1 + e^{\alpha\left(d(\mathbf{x}_k, \mathbf{p}_j) - r\right)}} \tag{12}$$

where $d(\mathbf{x}_k, \mathbf{p}_j) = \sqrt{(p_{x,k} - p_{j,x})^2 + (p_{y,k} - p_{j,y})^2 + \epsilon}$ with $\epsilon = 10^{-6}$ ensuring differentiability at zero distance. The numerically stable form $\frac{W}{2}\big(1 - \tanh\big(\frac{\alpha}{2}(d - r)\big)\big)$ is used in implementation to avoid overflow. Points beyond the search radius are replaced with far sentinels, keeping the NLP sparsity pattern constant and enabling warm starting. Here W is the barrier height, α controls steepness, and r is the safety radius.

4) Box Constraints and Control Bounds: Control inputs are bounded by:

$$-v_{x,\max} \leq v_{x,k} \leq v_{x,\max}, \quad -v_{y,\max} \leq v_{y,k} \leq v_{y,\max}, \quad -\omega_{\max} \leq \omega_k \leq \tag{13}$$

with nominal values $v_{x,\max} = 1.0\,\text{m/s}$, $v_{y,\max} = 0.5\,\text{m/s}$, $\omega_{\max} = 1.5\,\text{rad/s}$.

5) Reference Trajectory and Warm Starting: The reference sequence $\{\mathbf{x}_{\text{ref},k}\}_{k=0}^{N}$ is generated by advancing along the A* path at cruise speed v_{ref} from the arc-length coordinate s_0 of the closest waypoint to the robot:

$$s_k = \min(s_0 + v_{\text{ref}} \cdot k \cdot \Delta t,\ L_{\text{path}}) \tag{14}$$

Reference position $\mathbf{p}_{\text{ref},k}$ is linearly interpolated along the path segment containing s_k; reference yaw is the path tangent $\psi_{\text{ref},k} = \text{atan2}(\Delta y, \Delta x)$. The previous IPOPT solution is shifted by one step and used as the warm-start initial guess, substantially reducing solver iterations across consecutive ticks.

E. Setpoint Tracking Controller

A proportional controller running at 20 Hz converts the MPC lookahead setpoint into body-frame velocity commands. The setpoint is selected as the first predicted state at least $d_{\text{la}} = 2.0\,\text{m}$ ahead in the MPC horizon; in the near-goal regime, the final A* waypoint is used directly. The position error in the robot body frame is:

$$\begin{bmatrix} e_x \\ e_y \end{bmatrix}_{\text{body}} = \mathbf{R}(\psi)^\top \begin{bmatrix} s_x - p_x \\ s_y - p_y \end{bmatrix} \tag{15}$$

and the velocity commands are:

$$v_x^{\text{cmd}} = \text{clip}(k_{p,xy}\, e_x), \quad v_y^{\text{cmd}} = \text{clip}(k_{p,xy}\, e_y) \tag{16}$$

Optionally, when `enable_yaw_control` is active, a yaw error P-controller additionally computes $\omega^{\text{cmd}} = \text{clip}(k_{p,\psi}\, \text{wrap}(\psi_{\text{sp}} - \psi))$. A safety timeout zeroes all commands if no fresh setpoint is received within 2 s.

F. Offline Bayesian Optimization for MPC Tuning

1) Parameter Space: The MPC parameter vector $\theta \in \Theta \subset \mathbb{R}^{13}$ comprises: position tracking weights $Q_{xy} \in [50, 500]$, $Q_\psi \in [0.01, 2.0]$; terminal cost multiplier $Q_T \in [20, 300]$; control effort weights $R_{v_x}, R_{v_y}, R_\omega \in [0.1, 3.0]$; smoothness weight $R_j \in [0.1, 5.0]$; obstacle barrier height $W \in [50, 400]$, safety radius $r_{\text{obs}} \in [0.35, 0.85]\,\text{m}$, steepness $\alpha \in [1.0, 8.0]\,\text{m}^{-1}$; A* soft penalty $w_{\text{obs}} \in [10, 500]$; and lookahead distance $d_{\text{la}} \in [0.5, 2.5]\,\text{m}$. Each parameter is bounded in a physics-informed interval derived from prior hand-tuning, defining the convex search space $\Theta = \prod_i [\theta_i^{\min}, \theta_i^{\max}]$.

2) Composite Performance Score: Each function evaluation requires launching a full Gazebo simulation (5–8 min per trial), making gradient-free, sample-efficient methods essential. The composite score $\mathcal{J}(\theta)$ aggregates three navigation scenarios $s \in \{s_{\text{open}}, s_{\text{diag}}, s_{\text{lat}}\}$ (forward straight-line, diagonal, pure lateral) with scenario weights w_s:

$$\mathcal{J}(\theta) = \sum_s w_s \cdot \mathcal{J}_s(\theta), \qquad \sum_s w_s = 1 \tag{17}$$

Each per-scenario score $\mathcal{J}_s$ integrates five metrics. Let $\mathbf{p}_0$, $\mathbf{p}_f$, and $\mathbf{g}$ denote the robot start, final, and goal positions. *Goal progress* $\phi = \max(0, (d_0 - d_f)/d_0)$ measures the fraction of initial distance closed, where $d_0 = \|\mathbf{p}_0 - \mathbf{g}\|$, $d_f = \|\mathbf{p}_f - \mathbf{g}\|$. *Path efficiency* $\eta = \min(1, d_0/L)$ is the ratio of the straight-line distance to the total traversed arc length L. *Control smoothness* $s = e^{-\bar{j}/2}$ uses the exponentiated negative mean second-order finite difference of the command sequence as a jerk proxy:

$$\bar{j} = \frac{1}{K-2} \sum_{k=1}^{K-2} |\Delta^2 \mathbf{u}_k|_1, \quad \Delta^2 \mathbf{u}_k = \mathbf{u}_{k+1} - 2\mathbf{u}_k + \mathbf{u}_{k-1} \tag{18}$$

Obstacle avoidance $\mathcal{O}$ uses per-scan minimum LiDAR distances $\{d_k^{\text{obs}}\}$ to define danger ($d < 0.3\,\text{m}$) and warning ($0.3 \le d < 0.6\,\text{m}$) zone fractions:

$$\mathcal{O} = 0.50\,(1 - f_{\text{danger}}) + 0.30\,(1 - f_{\text{warning}}) + 0.20\,\min\left(\frac{\bar{d}}{2}, 1\right) \tag{19}$$

where $\bar{d}$ is the mean clearance capped at 2 m. *Time efficiency* $\tau = \min(1, T_{\text{ref}}/T_{\text{elapsed}})$ with $T_{\text{ref}} = d_0/0.5$ is only defined when the goal is reached.

Two scoring branches are defined:

$$\mathcal{J}_s = \begin{cases} 0.35 + 0.20\,\eta + 0.15\,s + 0.20\,\mathcal{O} + 0.10\,\tau & \text{if goal reached} \\ 0.25\,\phi + 0.10\,\eta + 0.10\,s + 0.15\,\mathcal{O} & \text{if goal not reached} \end{cases} \tag{20}$$

The 0.40 gap between the maximum scores of the two branches creates a strong gradient toward goal completion while preserving discriminative signal among failing parameter sets.

3) Tree-Structured Parzen Estimator: The offline BO employs Tree-Structured Parzen Estimators (TPE) [6], well-suited for high-dimensional mixed parameter spaces. TPE partitions the observed scores at the $\gamma = 0.15$ quantile threshold y^* and models two marginal densities:

$$\ell(\theta) = p(\theta \mid \mathcal{J}(\theta) < y^*), \quad g(\theta) = p(\theta \mid \mathcal{J}(\theta) \ge y^*) \tag{21}$$

Each marginal is estimated as a mixture of truncated Gaussians, one centred on each past observation. The acquisition is the Expected Improvement ratio:

$$\text{EI}(\theta) \propto \frac{\ell(\theta)}{g(\theta)} \tag{22}$$

The algorithm first draws $N_{\text{rand}} = 8$ trials uniformly (cold start), then iterates: refit ℓ and g, sample 25 candidates from $\ell(\theta)$, select the one with highest ℓ/g ratio, evaluate $\mathcal{J}$, and add to the dataset. This repeats until $N_{\max} = 30$ trials. The best observed $\theta^*_{\text{offline}} = \arg\max_i \mathcal{J}(\theta_i)$ is deployed as the base configuration.

4) GP Surrogate for Sensitivity Analysis: An ARD Matérn-5/2 Gaussian Process is fitted to all accumulated observations for interpretability. The kernel is:

$$k(\theta, \theta') = \sigma_f^2 \left(1 + \sqrt{5}\,r + \tfrac{5}{3} r^2\right) e^{-\sqrt{5}\,r} + \sigma_n^2 \delta(\theta, \theta') \tag{23}$$

where $r = \sqrt{\sum_i (\theta_i - \theta_i')^2/\ell_i^2}$ and ARD length scales $\{\ell_i\}$ are learned by maximizing the log marginal likelihood. The normalized inverse length-scale $\tilde{\ell}_i = (1/\ell_i)/\sum_j (1/\ell_j)$ quantifies parameter sensitivity: a short ℓ_i indicates that the score landscape varies rapidly with θ_i, marking it as a critical parameter.

G. Online Bayesian Optimization

The offline-tuned $\theta^*_{\text{offline}}$ provides a strong initialization but cannot account for characteristics of specific deployment environments not represented in the training suite. The online BO pipeline addresses this by maintaining a GP surrogate updated incrementally after each navigation episode.

1) Online Performance Signal: During deployment, the composite score $\mathcal{J}_{\text{online}}$ is computed in real time from ROS 2 topic streams: position error from `/go2/pose`, obstacle proximity from `/lidar/points_filtered`, and control smoothness from `/cmd_vel`. The same five-metric formulation of (20) is applied, providing a consistent scoring interface between offline simulation and online hardware.

2) Incremental GP Update and EI Acquisition: Let $\mathcal{D}_n = \{(\theta_i, \mathcal{J}_i)\}_{i=1}^n$ be the observation set accumulated during deployment, initialized with $\mathcal{D}_0 = \{(\theta^*_{\text{offline}}, \mathcal{J}(\theta^*_{\text{offline}}))\}$. The GP posterior after n observations is:

$$\mu_n(\theta) = \mathbf{k}_n(\theta)^\top (K_n + \sigma_n^2 I)^{-1} \mathbf{y}_n \tag{24}$$

$$\sigma_n^2(\theta) = k(\theta,\theta) - \mathbf{k}_n(\theta)^\top (K_n + \sigma_n^2 I)^{-1} \mathbf{k}_n(\theta) \tag{25}$$

where $K_n \in \mathbb{R}^{n\times n}$ is the kernel matrix and $\mathbf{k}_n(\theta) \in \mathbb{R}^n$ is the covariance vector with training points. The next query is selected by maximizing the closed-form Expected Improvement:

$$\text{EI}(\theta) = (\mu_n(\theta) - \mathcal{J}_n^*)\Phi(Z) + \sigma_n(\theta)\phi(Z), \quad Z = \frac{\mu_n(\theta) - \mathcal{J}_n^*}{\sigma_n(\theta)} \tag{26}$$

where $\mathcal{J}_n^* = \max_{i\le n} \mathcal{J}_i$, Φ and ϕ are the standard normal CDF and PDF respectively, and the acquisition is maximized numerically over Θ. To bound computational cost in the online setting, the GP is maintained on a sliding window of the $N_{\text{win}} = 20$ most recent observations, and kernel hyperparameters are re-optimized via marginal likelihood maximization every 5 episodes. The proposed $\theta_{n+1} = \arg\max_\theta \text{EI}(\theta)$ is applied for the next episode, and the resulting score augments $\mathcal{D}_{n+1}$, closing the adaptation loop. This mechanism enables the robot to progressively discover parameter settings matched to the specific deployment environment — particularly critical for out-of-distribution scenarios such as E4 and E5 in our evaluation.

H. GNN-Based Environment Embedding for Adaptive MPC

1) Problem Formulation: The BO pipelines adapt θ globally across navigation episodes. To enable *within-episode*, scan-by-scan parameter conditioning, we train a point cloud encoder $f_\phi : \mathbb{R}^{N\times 3} \to \mathbb{R}^d$ that maps the current LiDAR observation $\mathcal{P}_k$ to a compact, rotation-invariant scene embedding $\mathbf{z}_k = f_\phi(\mathcal{P}_k) \in \mathbb{R}^d$. A lightweight MLP $g_\psi : \mathbb{R}^d \to \mathbb{R}^{n_\theta}$ then predicts a parameter residual $\Delta\theta_k = g_\psi(\mathbf{z}_k)$, added to the BO-tuned base:

$$\theta_k = \text{clip}(\theta^* + \Delta\theta_k,\ \Theta) \tag{27}$$

This formulation decouples environment-agnostic base tuning (BO) from scan-level scene-specific adjustment (GNN), allowing independent development and validation of each component.

2) Point Cloud Preprocessing: Raw LiDAR scans have non-uniform point density. Farthest Point Sampling (FPS) [**?**] selects $K = 1024$ spatially uniform representative points iteratively:

$$p_1 = \text{random}, \qquad p_{j+1} = \arg\max_{p\in\mathcal{P}} \min_{i\le j} \|p - p_i\|_2 \tag{28}$$

The sampled cloud is then normalized to the unit sphere to remove absolute position and scale:

$$\tilde{\mathbf{p}}_i = \frac{\mathbf{p}_i - \bar{\mathbf{p}}}{\max_j \|\mathbf{p}_j - \bar{\mathbf{p}}\|_2}, \qquad \bar{\mathbf{p}} = \tfrac{1}{K}\textstyle\sum_i \mathbf{p}_i \tag{29}$$

making the embedding invariant to absolute sensor position and environment scale.

3) Dynamic Graph CNN Architecture: The encoder f_ϕ is a Dynamic Graph CNN (DGCNN) [**?**] with three stacked EdgeConv layers. At each layer ℓ, a directed k-NN graph $\mathcal{G}^{(\ell)}$ is constructed *dynamically* in the current feature space, not fixed in 3-D input space. This allows early layers to capture local 3-D geometry while deeper layers group semantically related regions. For each directed edge $(i \to j)$, an edge feature is computed by a shared MLP $h_\Theta^{(\ell)}$:

$$\mathbf{e}_{ij}^{(\ell)} = h_\Theta^{(\ell)}\Big([\mathbf{h}_i^{(\ell)} \,\|\, \mathbf{h}_j^{(\ell)} - \mathbf{h}_i^{(\ell)}]\Big) \tag{30}$$

The concatenation of $\mathbf{h}_i^{(\ell)}$ (global shape context) and $\mathbf{h}_j^{(\ell)} - \mathbf{h}_i^{(\ell)}$ (relative local geometry) is the key inductive bias: it is invariant to global translation while sensitive to local structure. Node features are updated by max-pooling over neighbors:

$$\mathbf{h}_i^{(\ell+1)} = \max_{j\in\mathcal{N}^{(\ell)}(i)} \mathbf{e}_{ij}^{(\ell)} \tag{31}$$

After three EdgeConv layers (channel dimensions 64, 128, 256), multi-scale features are aggregated via skip concatenation:

$$\mathbf{F} = [\mathbf{F}^{(1)} \,\|\, \mathbf{F}^{(2)} \,\|\, \mathbf{F}^{(3)}] \in \mathbb{R}^{N\times 448} \tag{32}$$

A per-point MLP maps to $\mathbb{R}^{512}$, then symmetric global pooling collapses the point dimension:

$$\mathbf{g} = \max_i \mathbf{f}_i + \frac{1}{N}\sum_i \mathbf{f}_i \ \in\ \mathbb{R}^{512} \tag{33}$$

combining peak activations (max) with distributional statistics (mean). A final linear layer projects to $\mathbf{z} \in \mathbb{R}^d$ with $d = 256$. Total trainable parameters: $\approx 2.8\,\text{M}$.

4) Self-Supervised Training via Contrastive Learning: The encoder f_ϕ is trained without labels using the SimCLR framework [**?**] adapted for 3-D point clouds. Given a batch $\{\mathcal{P}_i\}_{i=1}^B$, two independent stochastic augmentations $t_1, t_2 \sim \mathcal{T}$ create positive pairs $(\tilde{\mathcal{P}}_i^{(1)}, \tilde{\mathcal{P}}_i^{(2)})$; all $2(B-1)$ cross-sample pairs are negatives. The augmentation pipeline includes random yaw rotation ($\theta_r \sim \mathcal{U}(0, 2\pi)$) for horizontal rotational invariance, isotropic scaling ($s \sim \mathcal{U}(0.8, 1.25)$), x-axis reflection ($p = 0.5$), and Gaussian jitter ($\sigma_j = 0.01$, clipped at ± 0.05) for sensor noise robustness.

Projection head outputs are L2-normalized: $\hat{\mathbf{z}} = \mathbf{z}/\|\mathbf{z}\|_2$. The symmetric InfoNCE (NT-Xent) loss for a positive pair (i, j) in the joint batch $\mathbf{Z} \in \mathbb{R}^{2B\times d}$ is:

$$\ell(i,j) = -\log \frac{\exp(\hat{\mathbf{z}}_i^\top \hat{\mathbf{z}}_j/\tau)}{\sum_{k=1}^{2B} \mathbf{1}_{[k\neq i]} \exp(\hat{\mathbf{z}}_i^\top \hat{\mathbf{z}}_k/\tau)} \tag{34}$$

$$\mathcal{L} = \frac{1}{2B}\sum_{i=1}^{B}[\ell(i, i{+}B) + \ell(i{+}B, i)] \tag{35}$$

with temperature $\tau = 0.07$. The InfoNCE loss provides a lower bound on mutual information between positive views [**?**]: $\mathcal{L}_{\text{NCE}} \geq \log(B) - I(v_1; v_2)$, so minimizing $\mathcal{L}$ maximizes $I(v_1; v_2)$ and forces the encoder to capture scene structure invariant to the applied augmentations. Training uses Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, LR $= 10^{-3}$) with cosine annealing to 10^{-5} over 100 epochs, gradient clipping at $\|\nabla\|_2 \leq 1.0$, and batch size 32. After training, the projection head is discarded; only f_ϕ is used for inference at ≈ 4 ms per scan.

IV. Experimental Setup

A. Robot Platform and Software Stack

All experiments are conducted on the Unitree Go2 quadruped. Locomotion is provided by the CHAMP hierarchical controller [2], exposing a body-frame `/cmd_vel` interface. A mid-range 3-D LiDAR provides filtered point clouds at 10 Hz. The navigation stack runs on an Intel NUC 12 (Core i7-1260P, 16 GB RAM) carried onboard, using ROS 2 Humble [**?**], CasADi 3.6 [**?**], and IPOPT 3.14 [**?**]. Simulation is performed in Gazebo Fortress [**?**] with a planar-move plugin providing ground-truth odometry. The GNN encoder is pre-trained on point clouds collected from all five environments in simulation, then deployed both in simulation and on hardware without fine-tuning.

B. Evaluation Environments

Five Gazebo worlds of increasing complexity are employed (Fig. 3). Environments E1–E3 are used for offline BO tuning; E4 and E5 are held out as out-of-distribution test scenarios.

- **E1 — Open Plain** (training): 10×10 m obstacle-free space; validates baseline trajectory tracking and cruise speed.
- **E2 — Narrow Corridor** (training): 8 m-long, 0.9 m-wide passageway; evaluates lateral clearance maintenance under the MPC barrier.
- **E3 — Cluttered Static** (training): 6×6 m arena with 12 randomly placed cylindrical obstacles ($r = 0.15$ m); tests dense static obstacle avoidance and path efficiency.
- **E4 — Dynamic Obstacle Field** (unseen): 8×8 m space with 4 obstacles moving at $v \in [0.3, 0.6]$ m/s; assesses generalization to non-static environments.
- **E5 — Complex Multi-Room** (unseen): L-shaped corridor network with narrow junctions ($w < 1.0$ m) and partial occlusion; tests topological and structural generalization.

Fig. 3. The five Gazebo evaluation environments. Top row (training): E1 Open Plain, E2 Narrow Corridor, E3 Cluttered Static. Bottom row (held-out): E4 Dynamic Obstacle Field, E5 Complex Multi-Room. Robot trajectories for the proposed method are overlaid.

C. Evaluation Protocol

Phase 1 — Offline BO: 30 TPE trials are run using only E1–E3, producing $\theta^*_{\text{offline}}$. Each trial runs three scenarios in Gazebo (open, diagonal, lateral) and computes the composite score (17).

Phase 2 — Generalization evaluation: All five configurations are evaluated with *fixed* parameters (no online update) across all five environments. This assesses how well the offline-tuned parameters transfer to unseen scenarios.

Phase 3 — Online BO: Starting from $\theta^*_{\text{offline}}$, the GP-EI online loop runs for 15 episodes per environment. On E1–E3 this measures incremental improvement over the already well-tuned offline solution; on E4–E5 it measures adaptation to out-of-distribution environments from a warm-started prior.

Phase 4 — GNN conditioning: The DGCNN encoder is integrated and the full Online BO + GNN pipeline is evaluated across all five environments.

Each configuration is evaluated over 50 independent navigation episodes per environment (goal distance 4–8 m, random start orientation). We report: *Success Rate* (SR, %): fraction of episodes reaching the goal within 0.5 m without collision; *Cross-Track RMSE* (CE, m): root mean square lateral deviation from the reference path; *Collision Rate* (CR, %): fraction of episodes with at least one obstacle contact (clearance $<$ 0.15 m); and *Path Efficiency* η: ratio of optimal to actual path length, averaged over successful episodes.

V. Results

A. Navigation Performance

Tables I and II report the full performance comparison. Fig. 4 shows the offline BO convergence and online adaptation trajectory. Fig. 5 shows representative navigation trajectories in E3.

The fixed MPC baseline achieves satisfactory performance in the open environment (E1, SR = 88%) but degrades markedly in constrained and dynamic scenarios (E4: SR = 48%, CR = 31%). Offline BO on E1–E3 consistently improves all metrics, with success rates increasing by up to 22 percentage points in the corridor scenario (E2: 62% → 84%) and generalizing meaningfully to unseen environments (E4: 48% → 64%, E5: 44% → 59%), demonstrating that optimizing across diverse training scenarios provides positive transfer.

Online BO provides additional gains concentrated in the out-of-distribution environments: E4 improves from 64% to 76%, and E5 from 59% to 73%, as the GP-EI loop identifies environment-specific parameter adjustments not covered by the offline training distribution. These gains are measurable within the first 10 online episodes, owing to the warm-started GP that leverages the offline prior to direct exploration efficiently.

GNN conditioning (Offline BO + GNN) yields gains comparable to Online BO on in-distribution environments, confirming that scan-level parameter adaptation captures real-time geometric context relevant for performance. The full proposed pipeline (Online BO + GNN) achieves the highest performance across all scenarios: 98% SR in E1, 91% in E2, 93% in E3, 85% in E4, and 82% in E5, with cross-track RMSE below 0.10 m in the open environment and below 0.25 m in the most challenging unseen scenario. The collision rate is reduced to zero for in-distribution environments and to 8–11% for unseen dynamic scenarios.

B. Parameter Sensitivity

Fig. 6 reports the GP-derived parameter sensitivity after 30 offline BO trials. The obstacle barrier weight W and position tracking weight Q_{xy} exhibit the highest sensitivity ($\tilde{\ell} > 0.20$), confirming that MPC performance is most critically determined by the balance between trajectory tracking and obstacle avoidance. The yaw weight Q_ψ and jerk weight R_j show low sensitivity ($\tilde{\ell} < 0.05$), indicating robustness to their values within the searched range.

C. Computational Overhead

Table III reports the computational cost of each module. The core MPC solve time remains below 20 ms, comfortably within the 100 ms budget of the 10 Hz control loop. GNN inference adds ≈4 ms per tick via a compiled forward pass. The online GP update, executed asynchronously every 5 episodes, completes in <50 ms on the onboard NUC and does not interfere with the real-time control loop.

Fig. 4. Left: offline BO convergence — aggregate score $\mathcal{J}(\theta)$ vs. trial index over 30 trials (shaded region: $\pm 1\sigma$ across scenarios). The dashed vertical line separates the random initialization phase from TPE-guided search. Right: online BO score evolution during deployment on E4 (dynamic, unseen), averaged over 15 episodes. The online GP rapidly converges beyond the offline initialization (dashed horizontal line).

VI. Discussion and Conclusion

We have presented a general-purpose autonomous navigation framework integrating reactive rolling-horizon planning with dual-stage adaptive nonlinear MPC, augmented by offline and online Bayesian Optimization and self-supervised GNN-based environment embedding. The system achieves consistent performance improvement across a five-environment ablation, reaching up to 98% success rate in simulation and confirming real-world applicability through hardware deployment on the Unitree Go2 quadruped with negligible sim-to-real degradation.

A key finding is the complementary nature of the three adaptation mechanisms. Offline BO provides a broadly generalizing initialization by optimizing across diverse training scenarios. Online BO enables rapid, environment-specific refinement within as few as 10 deployment episodes via warm-started GP posterior updates, proving most valuable on out-of-distribution environments. GNN conditioning adds a within-episode layer orthogonal to episode-level BO, reducing cross-track error and collision rate without significant computational overhead. The modular architecture makes each layer independently deployable, offering practitioners a flexible performance-cost trade-off.

Limitations. The current kinematic model assumes relatively flat terrain [**?**]; significant slopes or steps would require extension to 3-D dynamics. The sliding-window GP scales linearly in the window size but may lose memory of early observations in very long deployments [**?**]. The GNN encoder

TABLE I
NAVIGATION PERFORMANCE: SUCCESS RATE (SR, %) AND CROSS-TRACK RMSE (CE, M). BOLD: BEST PER COLUMN. E1–E3: IN-DISTRIBUTION TRAINING ENVIRONMENTS; E4–E5: HELD-OUT OUT-OF-DISTRIBUTION ENVIRONMENTS.

	E1 (Open)		**E2 (Corridor)**		**E3 (Cluttered)**		**E4 (Dynamic)**		**E5 (Multi-Room)**	
Method	SR	CE	SR	CE	SR	CE	SR	CE	SR	CE
Fixed MPC	88	0.19	62	0.34	67	0.29	48	0.44	44	0.48
Offline BO	95	0.12	84	0.22	88	0.19	64	0.35	59	0.40
Online BO	97	0.10	87	0.17	90	0.15	76	0.26	73	0.30
Offline BO + GNN	96	0.11	84	0.19	88	0.17	71	0.29	68	0.33
Online BO + GNN	**98**	**0.08**	**91**	**0.14**	**93**	**0.12**	**85**	**0.20**	**82**	**0.24**

TABLE II
COLLISION RATE (CR, %) AND PATH EFFICIENCY (η). BOLD: BEST PER COLUMN.

	E1 (Open)		**E2 (Corridor)**		**E3 (Cluttered)**		**E4 (Dynamic)**		**E5 (Multi-Room)**	
Method	CR	η	CR	η	CR	η	CR	η	CR	η
Fixed MPC	3	0.91	22	0.69	17	0.73	31	0.59	36	0.55
Offline BO	0	0.95	10	0.82	8	0.86	21	0.70	26	0.65
Online BO	0	0.96	6	0.86	4	0.89	13	0.79	17	0.77
Offline BO + GNN	0	0.95	8	0.85	6	0.88	16	0.77	20	0.74
Online BO + GNN	**0**	**0.97**	**3**	**0.90**	**2**	**0.92**	**8**	**0.86**	**11**	**0.83**

TABLE III
MEAN COMPUTATIONAL COST PER 10 HZ CONTROL TICK (INTEL NUC 12).

Module	**Time (ms)**
Gaussian Grid Rebuild	2.1 ± 0.4
A* Planning	3.8 ± 1.2
MPC Solve (IPOPT)	15.3 ± 3.4
GNN Inference (DGCNN)	4.2 ± 0.8
Setpoint Controller	0.3 ± 0.1
Total (with GNN)	$\mathbf{25.7 \pm 4.1}$

Fig. 5. Representative navigation trajectories in E3 (Cluttered Static). Fixed MPC (red): jerky maneuvers and near-collision events. Offline BO (orange): smoother paths with improved clearance. Online BO + GNN (green, proposed): consistent, efficient trajectories closely following the A* reference (dashed) with maintained obstacle clearance throughout.

was trained on structured indoor environments; performance on radically different distributions may require domain adaptation techniques [**?**].

Future Work. The framework is intentionally platform-agnostic, and we plan several extensions. First, the navigation stack will be ported to the Unitree H1 bipedal humanoid robot [**?**], where the holonomic velocity abstraction remains valid at the navigation layer but the locomotion controller must handle tighter stability constraints and reduced lateral velocity limits. The same BO and GNN adaptation pipelines apply without modification, offering a direct path from quadruped to humanoid deployment. Second, the framework will be interfaced with Visual Language Action (VLA) models [**?**], enabling natural-language task specification such as "bring the object from the desk to the table," decomposed by the VLA into navigation sub-goals that our stack executes autonomously. This interface is envisioned for both legged platforms (quadruped dog, humanoid) and wheeled agents, providing a unified navigation primitive callable from high-level multimodal reasoning systems. Third, we plan to extend the online BO to a lifelong continual learning setting [**?**] that accumulates knowledge across deployments without catastrophic forgetting, building a library of environment-specific parameter priors retrievable upon revisiting known scene types.

Fig. 6. GP-derived parameter sensitivity $\tilde{\ell}_i$ after 30 offline BO trials. The obstacle barrier height W and position tracking weight Q_{xy} dominate, confirming that the tracking-vs-safety trade-off is the primary determinant of navigation performance in the 13-dimensional parameter space.

REFERENCES

[1] H. Shah, E. Tevere, D. Atha, M. Kaufmann, S. Khattak, M. Patel, M. Hutter, J. Frey, and P. Spieler, "Wildos: Open-vocabulary object search in the wild," 2026. [Online]. Available: https://arxiv.org/abs/2602.19308

[2] L. Technology, "Hierarchical controller for highly dynamic locomotion utilizing pattern modulation and impedance control : implementation on the mit cheetah robot," *MIT Libraries (DSpace@MIT)*, 03 2014.

[3] T. F. Banfi, F. Dorati, N. Manzoni, and J. Martínez-Gómez, "Optimizing initial path finding in informed-rrt* with a novel map-adaptive sampling technique," in *2024 7th Iberian Robotics Conference (ROBOT)*, Nov 2024, pp. 1–6.

[4] D.-N. Bui, T. H. Khuat, M. D. Phung, T.-H. Tran, and D. L. Tran, "Model predictive control for optimal motion planning of unmanned aerial vehicles," in *2024 International Conference on Control, Robotics and Informatics (ICCRI)*, July 2024, pp. 1–6.

[5] A. G. Puigjaner, M. Prajapat, A. Carron, A. Krause, and M. N. Zeilinger, "Performance-driven constrained optimal auto-tuner for mpc," *IEEE Robotics and Automation Letters*, vol. 10, no. 5, pp. 4698–4705, May 2025.

[6] S. Watanabe, "Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance," 2025. [Online]. Available: https://arxiv.org/abs/2304.11127